

Building Autonomous Workflows with AI Agents

Service-Enabled Science — Session 05:
giving the agent tools, not just a chat window

Huihuo Zheng

Argonne Leadership Computing Facility

30 September 2026



Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility



U.S. DEPARTMENT
of ENERGY

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.

From services to agents



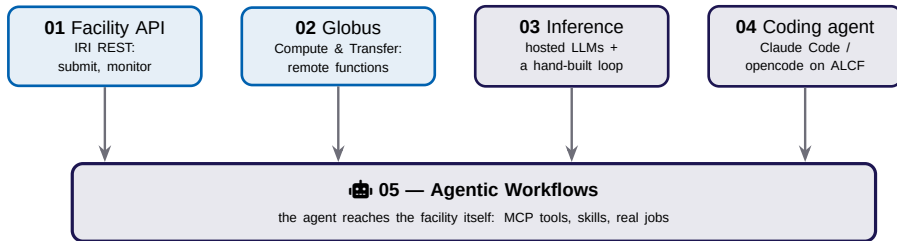
Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

Sessions 01–04 built the pieces

This session is where they get assembled into one agent



Same services, new caller. In 01–03 *you* made the calls; here the agent does — under your token, with you watching.

Accelerating scientific discovery through agentic automation

Where a scientist's time actually goes before the first result

>_ Manual workflow today

1. SSH to a login node
2. Read module docs, hand-write a job script
3. Fix account, queue, walltime, paths
4. Submit, poll qstat
5. Parse errors, edit, resubmit
6. Move output files by hand

Hours of mechanics before the science starts.

Agentic workflow

“Run my training script on 2 Polaris nodes and tell me when it converges.”

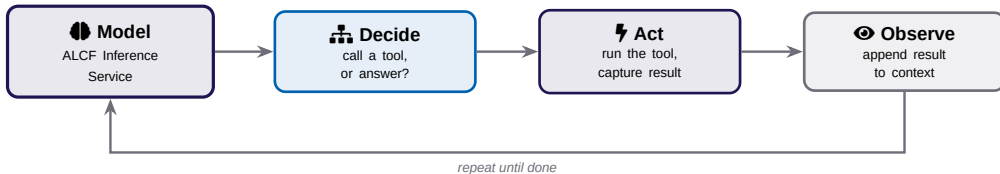
- Agent resolves account, queue, paths
- Stages the script, submits the job
- Monitors; diagnoses failures
- Reports results with evidence

The scientist stays in the loop and owns the science.

The mechanics are the bottleneck — and ALCF's service layer is what makes them programmable.

An agent is a loop

A model, a set of tools, and a stopping condition — nothing more



</> You already wrote one

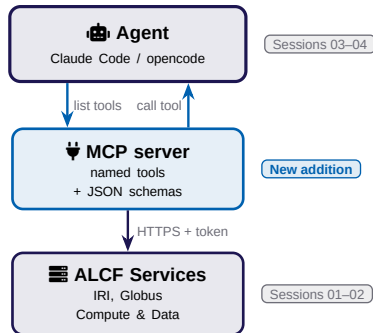
Session 03's `04_agent_loop.py` is exactly this loop in 198 lines, with one tool: `get_system_status`. Run it with `--step` to watch each turn.

🔧 What is missing is the tools

The loop is the easy part. The value is in *which* tools the model can reach — and whether they are safe to hand it.

Integrating ALCF services through MCP

Discovered at runtime, typed, and scoped by you



You built the top and bottom in Sessions 01–04. Today adds the middle.

? What is MCP (Model Context Protocol)?

- 🔌 **A standard interface** for an agent to discover and call a service's operations as typed tools.
- 🔍 **Discovered.** The agent asks the server what it can do — no endpoints, no hand-built payloads.
- ✅ **Typed.** A schema per tool, so bad arguments fail before a job runs.
- 🔧 **Configured once.** One `.mcp.json` entry, used by Claude Code and opencode alike.

Agentic Workflows building blocks



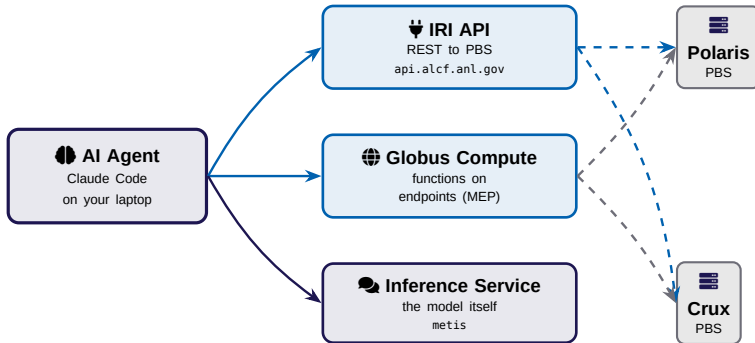
Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

Every service today is an agent tool

The same endpoints from Sessions 01–03, now reachable by name



One alcf-tokens login mints every token the agent needs — Globus OAuth2, short-lived, and yours.

Agent core: point your agent at the ALCF Inference Services

🔑 What each step does

1. **Authenticate once.**

Mints the inference, IRI and Globus tokens together.

2. **Configure the agent.**

Writes `~/.claude/settings.json`.

3. **Launch it.**

🔄 Re-run step 2 to refresh an expired key.

👤 **Already on Anthropic or Argo?** Skip step 2 — your existing setup keeps working.

🔧 **Otherwise back up first:** step 2 rewrites the endpoint, model and key entries in `~/.claude/settings.json` and `~/.config/opcode/opcode.jsonc`.

>_ setup

```
alcf-tokens login \  
  --authorize-transfer home \  
  --authorize-transfer eagle  
uvx alcf-ai agent configure claude  
# ... or: configure opcode  
claude
```

`./setup.sh` installs the CLIs and the Python env — step 1 is still yours.

🔧 Hands-on 3 also needs **Globus Connect Personal**. It wants a browser login, so install it *before* the session.

AskALCF — ALCF Knowledge Base

Grounding your response on facility user guides

 `alcf_tools/docs.py --- no token`

```
ASK = "https://ask.alcf.anl.gov/mcp"

@mcp.tool()
async def retrieve_alcf_docs(query, top_k=3):
    async with Client(ASK) as upstream:
        ...
```

The URL is the whole configuration — nothing to install, nothing to authenticate.

 Cloudflare 403s some clients by TLS fingerprint. Reaching it from *your* server — one that is also a *client* of another — fixes that once, for every client.

🔍 One tool, and why it earns its place

- 📖 `retrieve_alcf_docs(query, top_k)` — ALCF, OLCF, NERSC and LLNL guides, plus PBS, Slurm and CUDA.
- 🔗 Returns passages **with source URLs**, so you can check what the model was actually told.
- ❓ *“Please generate a submission script on Polaris.”* — PBS Pro directives written from the documented queue policy, not from the model’s memory.

ALCF IRI for compute

It wraps the same Session-01 REST calls you made by hand

 `alcf_mcp.py` --- one tool

```
@mcp.tool()
def submit_job(system: str, commands: str,
               stdout_path: str, nodes=1,
               queue="debug",
               account="alcf_training",
               walltime_sec=600) -> dict:
    """Submit a PBS job, return its ID.

    Refuses accounts outside the
    workshop allocation, >2 nodes, or
    walltimes over 30 minutes.
    """
```

The docstring is the tool description the model reads.

☰ Six IRI tools

- 📡 `get_system_status` — Polaris / Crux up?
- ⚡ `submit_job` — PBS job via IRI
- 🔄 `get_job_state` — poll one job
- ☰ `list_jobs` — filter by state / queue
- ✕ `cancel_job` — stop a run
- 📁 `read_file` — fetch stdout / stderr

Six REST endpoints from `01_Facility_API/`, now callable by name — and four more on the next slide.

Globus data transfer MCP

Sessions 01–04 only touched files already at the facility

alcf_mcp.py --- staging

```
@mcp.tool()
def stage_to_alcf(local_path: str,
                  alcf_path: str,
                  recursive=False) -> dict:
    """Copy a local file to ALCF.

    Returns as soon as Globus accepts
    the task -- the bytes have NOT
    moved yet. Poll transfer_status
    until SUCCEEDED before submitting.
    """
```

≡ The other three

- 🔌 `local_endpoint` — your GCP collection ID, so no UUID sits in the repo
- 📁 `globus_ls` — local or alcf; look before you invent a path
- 🔄 `transfer_status` — poll to SUCCEEDED

🔒 Guardrails, before the call

Eagle writes only under `/eagle/alcf_training/`; 500 files; 1 GiB.

Capture the chain: the submit-job skill

Generate, stage, submit, monitor, report — written down once

skills/submit-job/SKILL.md

```
---
name: submit-job
description: Generate, stage over Globus,
  submit via IRI, monitor, report back.
---
1. Generate the input locally; show it.
2. stage_to_alcf -> /eagle/<project>/
  Wait for SUCCEEDED. Never assume.
3. get_system_status, then submit_job.
4. Poll with backoff: 10s, then 30s.
5. Quote the stderr line, propose a fix,
  never resubmit on your own.
```

 what is a skill?

- ≡ **A repeatable workflow, written down** — the steps you would otherwise retype every session.
-  **Just a markdown file** — it diffs and ships like any other artifact.
-  **The description routes it** — that line is how the agent decides to load it.

One file spans the whole chain — the skill is where it lives, not the prompt.

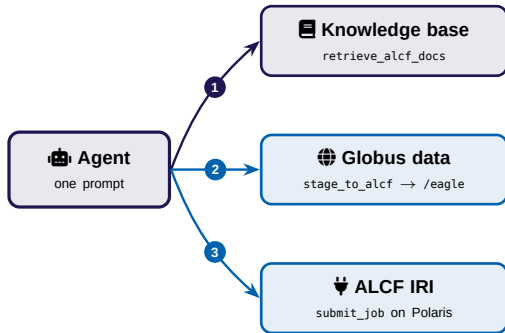
One ALCF MCP server, three tool groups for job submission

Ask the policy, stage the data, then submit

one server, three modules

```
alcf_mcp.py    mounts all three
alcf_tools/
__init__.py    the tool map
common.py      IDs, paths, guardrails
docs.py        1 tool    knowledge base
globus.py      4 tools   data transfer
iri.py         6 tools   compute
skills/
submit-job/SKILL.md  the chain
```

One entry in .mcp.json — each group keeps its own file.



Step 1 settles queue and walltime; step 3 reads what step 2 staged.

Three tool groups, one context — the agent carries the staged path and the job ID between them.

Hands-on



Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

Step 1: agent core and credentials

One clone, one login, one configure — then the agent is yours

>_ run these in order

```
git clone
↳ https://github.com/argonne-lcf/Service_Enabled_Science.git
cd Service_Enabled_Science
./setup.sh
source .venv/bin/activate
alcf-tokens login \
  --authorize-transfer home \
  --authorize-transfer eagle
alcf-tokens test-token inference
# {"ready": true, "error": null}
uvx alcf-ai agent configure claude
# ...or: configure opencode
```

i what each line buys you

-  `setup.sh` — installs uv, the .venv, opencode and Claude Code.
-  `login` — in the browser, pick the **Argonne LCF** provider; paste the code back.
-  `test-token` — the only proof the token works. Do it before anything else fails confusingly.
-  `configure` — points the agent at the ALCF Inference Service.

Step 2: Globus data transfer

A personal endpoint on your laptop, and a place to put files

> Linux --- download, set up, start

```
GCP=https://downloads.globus.org/globus-connect-personal/linux/stable
wget $GCP/globusconnectpersonal-latest.tgz && tar xzf globusconnect*.tgz
cd globusconnectpersonal-* && ./globusconnectpersonal # guided setup
./globusconnectpersonal -start &
```

✓ prove it works, then make somewhere to put files

```
./globusconnectpersonal -status # -> Globus Online: connected
ssh <you>@polaris.alcf.anl.gov 'mkdir -p /eagle/alcf_training/$USER'
```

🖥️ **macOS / Windows:** same base URL, mac/stable/...dmg or windows/stable/...exe — the installer's GUI does the same three steps.

⚠️ The default share is your *whole home directory*, read-write — narrow it in ~/.globusonline/lta/config-paths.

Step 3: first contact with your agent

Launch it and look around — before a single tool call

>_ start it

```
source .venv/bin/activate
claude
# [shift + tab] -> auto mode
```

🤖 then, in the session

```
> What model are you?
/model    # what is actually configured
/mcp      # the server and its 11 tools

> How many nodes does Polaris have?
What queues can I use?
```

On onepcode the equivalents are /models and /mcp.

👁 what to notice

- ❓ **Its answer is a guess.** Ask the system — /model — not the model.
- 🔌 **/mcp is the real check.** Not listed there, nothing later works.
- 📖 **The last one calls a tool.** retrieve_alcf_docs answers from live ALCF docs — no tool call means you got 2021.
- 👤 **Auto mode stops asking** — so the limits that matter live in the server.

Example 1: check system status

Read-only tools — nothing on this slide costs node-hours



"What's the status of Polaris and Crux right now? How many jobs are queued on Polaris, and what were the last five jobs to finish on Crux?"

`</>` the calls it makes --- watch which, and in what order

```
get_system_status(system="polaris")      # up / down, current reservations
get_system_status(system="crux")
list_jobs(system="polaris", states=["queued"])
list_jobs(system="crux", limit=5, historical=True)
```

what to watch for

It should *list* resources before naming one. An invented resource ID is the first failure mode.

try to break it

Ask about a system that does not exist. Ask for someone else's jobs. A good answer is "I can't tell you that."

Example 2: a two-node smoke test

The smallest job that proves the whole chain works

what you ask

Run a 2-node smoke test on Polaris under alcf_training, in reservation R7645913, that prints the hostname of every node, then show me the output when it lands.

the chain it runs

```
submit_job(system="polaris", nodes=2,  
            commands="mpiexec -n 2 --ppn 1 hostname",  
            queue="R7645913",           # the reservation  
            account="alcf_training", walltime_sec=600,  
            stdout_path="/home/<you>/smoke.out")  
get_job_state(system="polaris", job_id=...) # poll  
read_file(path="/home/<you>/smoke.out")
```

 In PBS a reservation *is* a queue name: R7645913 goes in queue=. Outside its window, use debug.

Plain hostname prints the head node *once* — two nodes allocated is not two nodes used; mpiexec -n 2 --ppn 1 makes the second answer. commands is the *work* only: the proxy and conda are prepended by the server (Example 3).

Ask for three nodes and the server says no — MAX_NODES = 2 lives in common.py, not in your prompt.

Example 3: train MNIST on Polaris

Stage the script with Globus, then train on a GPU node

 one prompt, all three tool groups

Stage mnist_pytorch.py to my eagle space, train on one Polaris node in reservation R7645913, and report the test accuracy.

 the chain it runs

```
stage_to_alcf("~/mnist_pytorch.py",  
  "/eagle/alc_f_training/<you>/")  
submit_job(system="polaris", nodes=1,  
  commands="python ../mnist_pytorch.py",  
  queue="R7645913", walltime_sec=1800,  
  stdout_path=".../mnist.out")  
get_job_state(...) # poll to completion  
read_file(".../mnist.out")
```

_ what the server prepends, so nobody forgets

```
export http_proxy=http://proxy.alcf.anl.gov:3128  
export https_proxy=http://proxy.alcf.anl.gov:3128  
module use /soft/modulefiles  
module load conda/2026-10-01
```

 Why it is code, not a prompt: every line fails *silently*. No proxy and MNIST's download hangs until walltime; no module use and the load is a no-op; unpinned and the Python changes under the room mid-workshop.

Example 4: running your own workload

Point the agent at your code — or ask it to build the software

 you already have the code

Stage ~/mycode to my eagle space,
build it on a Polaris compute node,
and run the 2-node test case.

 or you do not --- pick any code

Build LAMMPS (or QE, or CP2K) on
Polaris with GPU support, then run
its 2-node benchmark.

 what changes, what does not

- ≡ Same chain as before — generate, stage, submit, monitor, report. Only the payload is yours.
-  A build is just a job. Make it show you the compiler line it chose and the log it read.
-  The guardrails do not move: 2 nodes, 30 minutes, the training account.

This is the one you take home — the workload changes; the skill does not.

Where this goes



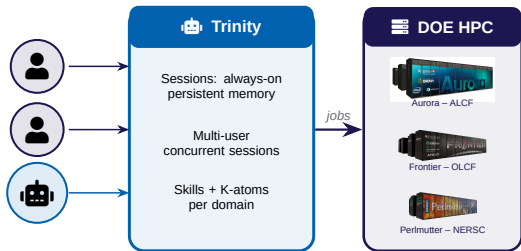
Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



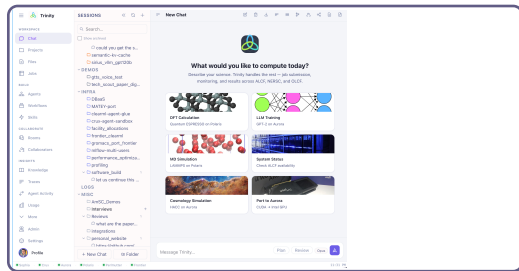
**Argonne Leadership
Computing Facility**

Where this goes

Serving HPC across DOE facilities via one Trinity platform



Serving HPC via AI agents — always-on, multi-user.



Trinity today: one chat interface, direct access to ALCF, NERSC, and OLCF.

 **Introducing Trinity** — trinityscience.org

Take it home

Everything in this session runs from your own laptop

Repositories

This workshop

argonne-lcf/Service_Enabled_Science

Deeper agent labs (ALCF)

zhenghh04/alcf_agentic_ai

Good next steps

-  Add one tool for *your* workload
-  Write one skill for what you retype weekly
-  Run the training lab: MNIST on Polaris
-  Tell us what broke — it is week one

The agent is the new layer. Everything underneath it you already used in Sessions 01–03.

Acknowledgement

This research used resources of the Argonne Leadership Computing Facility, a U.S. Department of Energy (DOE) Office of Science user facility at Argonne National Laboratory and is based on research supported by the U.S. DOE Office of Science-Advanced Scientific Computing Research Program, under Contract No. DE-AC02-06CH11357.

Thanks to the SES organizing team — Tom Uram, Christine Simpson, Benoit Côté, Michael Salim, Ethan Wong.

huihuo.zheng@anl.gov